2026 edition

Deep Learning for Music Analysis and Generation

Song Generation

(text → audio)



Yi-Hsuan Yang Ph.D.

yhyangtw@ntu.edu.tw

Song Generation: Intro

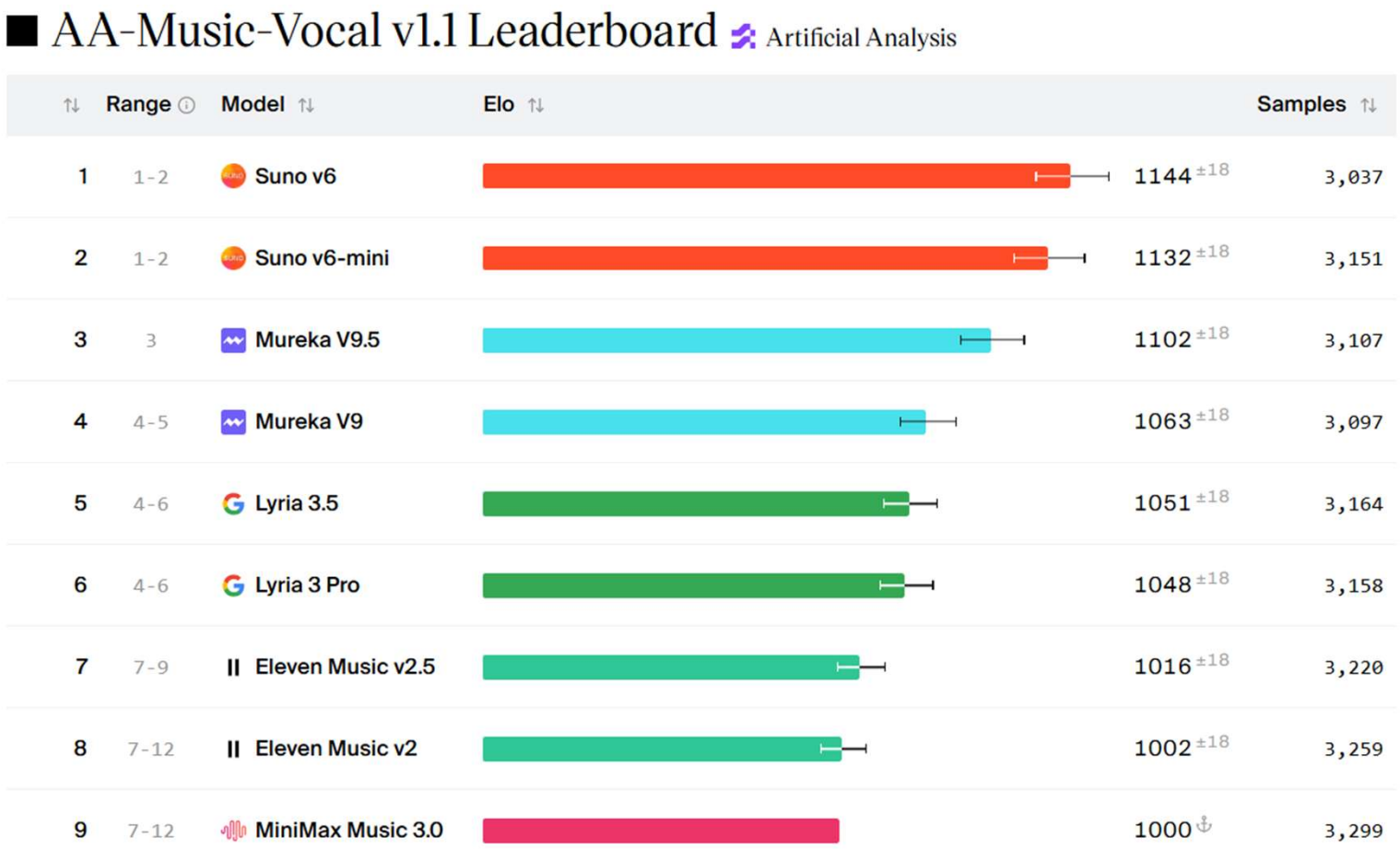
- Generate both **vocal** and the **instrumental** accompaniment
- Usually requires only **lyrics** and a style prompt (e.g., text) during inference
- Usually aim to generate music with **full-song length**
  - In contrast, SVS models usually generate one sentence at a time

- And… there might be copyright issues



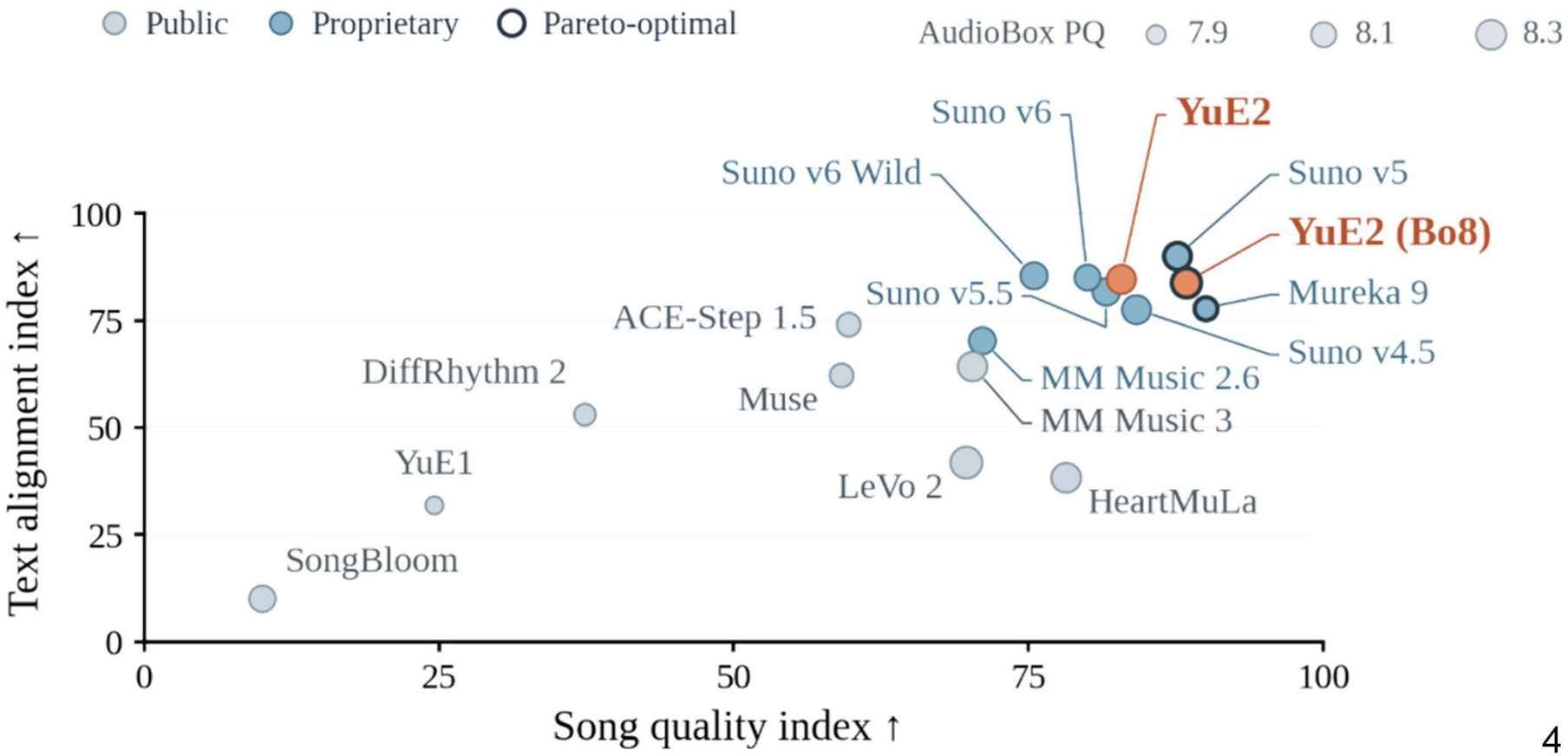
Leaderboard

https://huggingface.co/spaces/ArtificialAnalysis/Music-Arena-Leaderboard

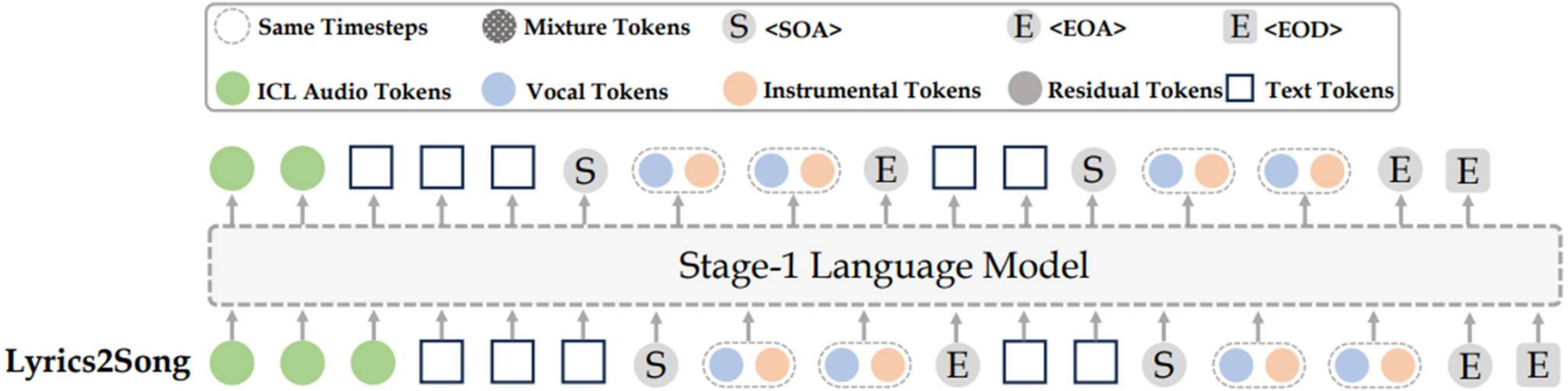


Open Source Models for Song Generation

- Quite a few since 2025; current SOTA: **YuE2** & **ACE-Step 1.5**



YuE

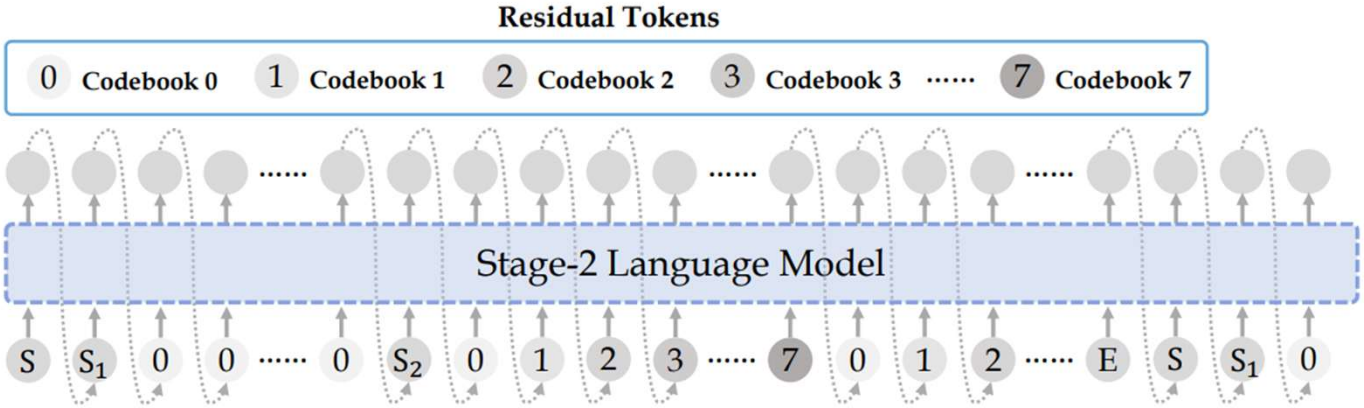


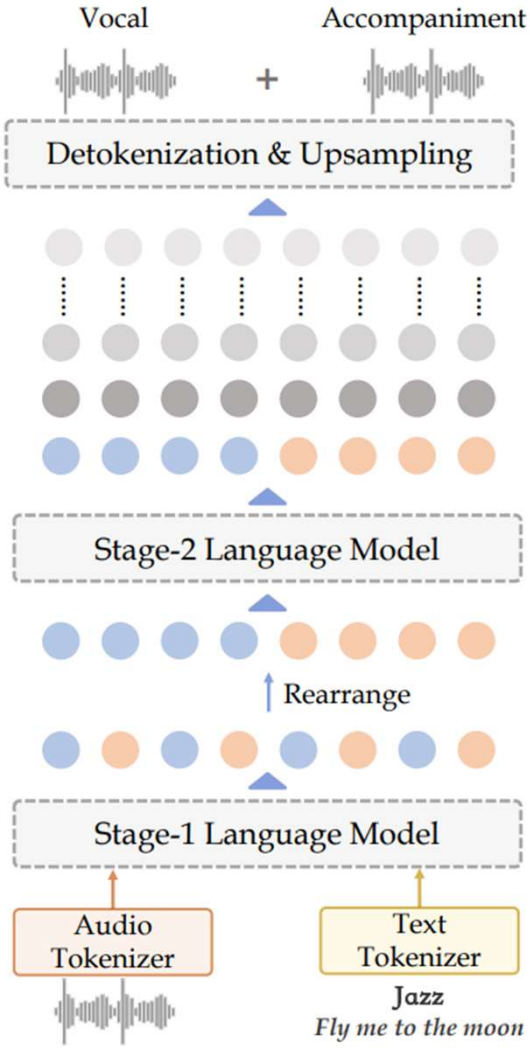
- **Sentence-by-sentence:** interleaving placement of lyrics and audio tokens
- **Codec**-based AR model: rely on attention to align lyrics and audio
- **Dual next token prediction**: vocal & instrumental tokens

Ref: Yuan et al, "YuE: Scaling open foundation models for long-form music generation," arXiv **2025.03** / ICLR 2026

YuE

- Two-stage LMs: coarse tokens first, then others
  - Both stages adopt the **LLaMA2** architecture
  - Use **X-codec** to facilitate lyrics conditioning
  - The training does not converge when using pure acoustic tokenizers such Encodec32k and HiFiCodec





Ref: Yuan et al, "YuE: Scaling open foundation models for long-form music generation," arXiv **2025.03** / ICLR 2026

YuE

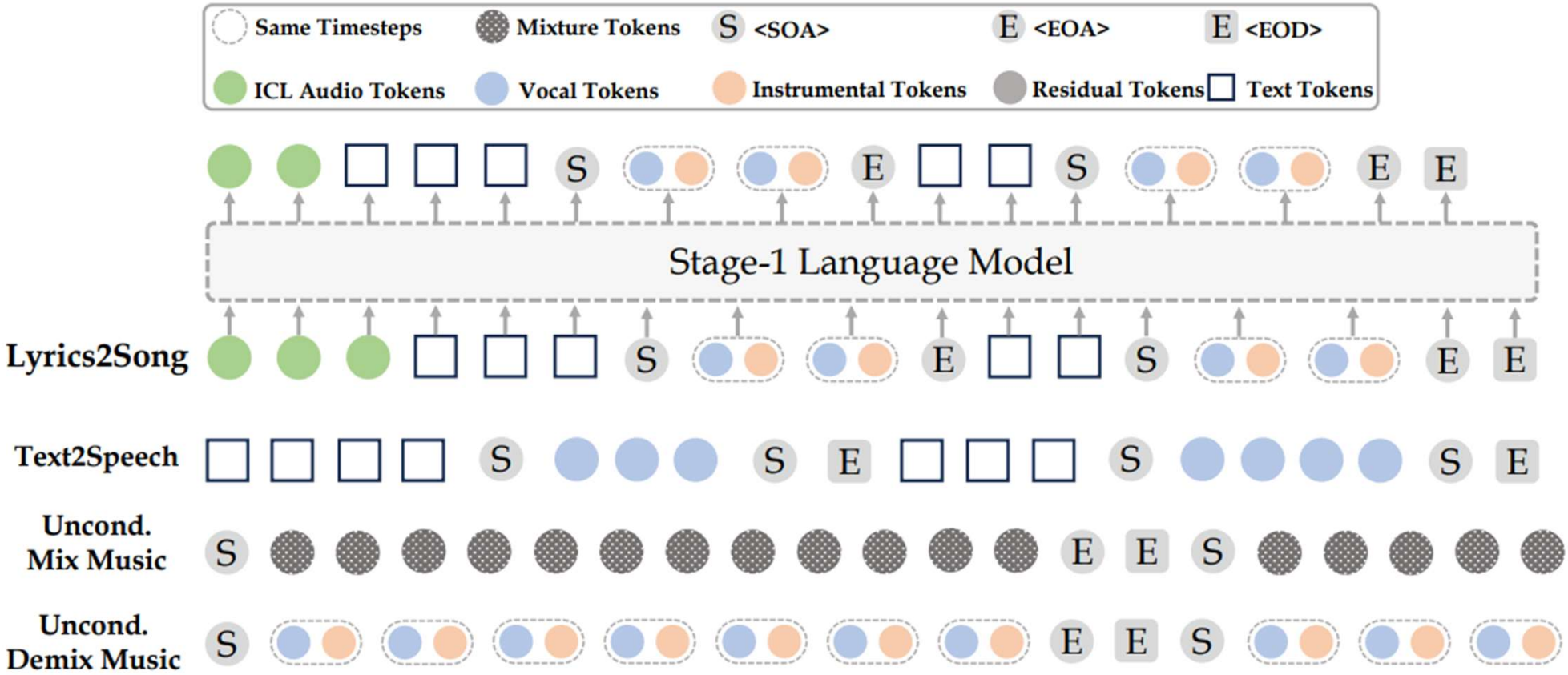
https://map-yue.github.io/

- Structural progressive conditioning for long-context lyrical alignment
- A multitask, multiphase pre-training recipe to converge and generalize
  - TTS
  - TTM
    - The majority of our dataset consists of unconditional music
    - We annotate all tracks using Qwen2-Audio
    - 40% tracks are separated into vocal-instrumental dual-track format using UVR12
  - Lyrics-to-song
    - we implement heuristic filtering to remove irrelevant content and exclude overly short lyrics (less than 10 sentences), retaining only approximately 10% of matched tracks

Table 2. Decomposition of Lyrics-to-Song Generation Capabilities

Essential Capabilities
1) Modeling of Human Vocal
2) Modeling of Instrumental
3) Joint Modeling of Vocal and Instrumental
4) Aligning Cross-Modal/Same-Modal Controls (lyrics, style, structure, in-context learning)

YuE

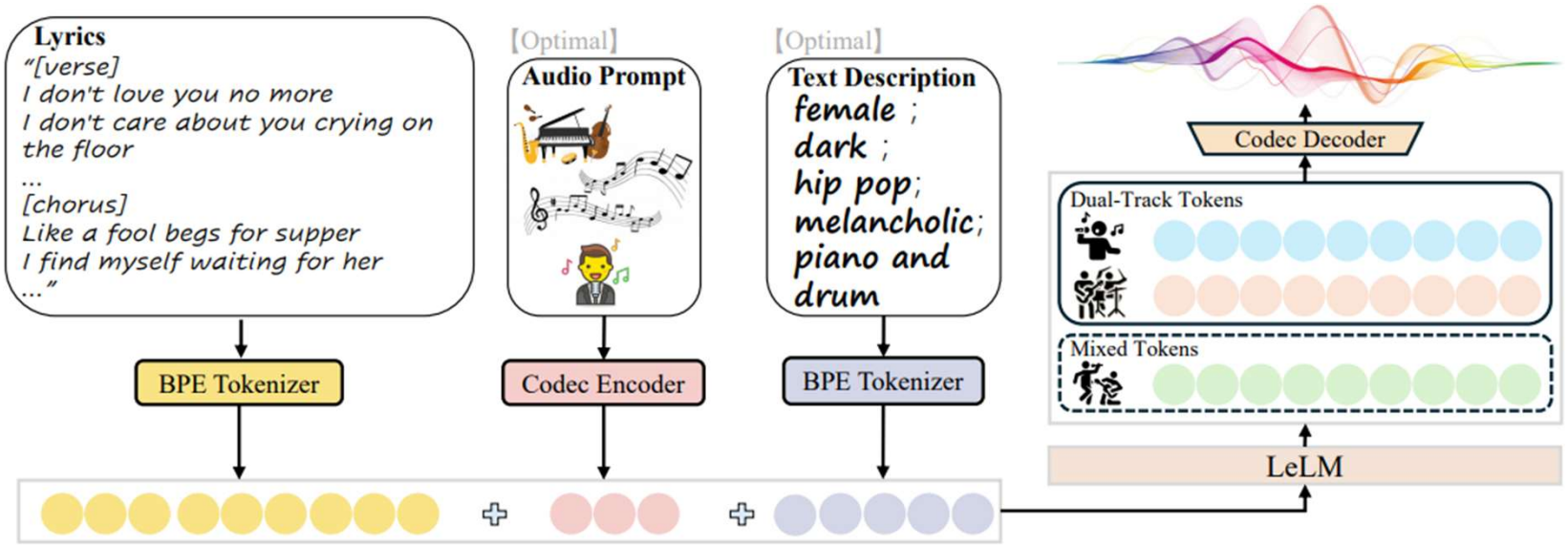


Ref: Yuan et al, "YuE: Scaling open foundation models for long-form music generation," arXiv **2025.03** / ICLR 2026

Levo

https://levo-demo.github.io/

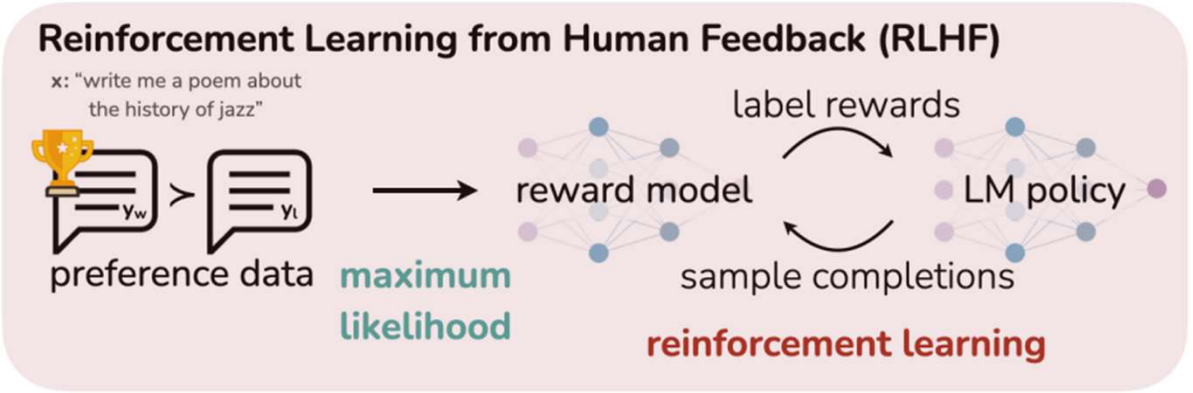
- Mixed tokens and dual-track tokens
- DPO (direct preference optimization)

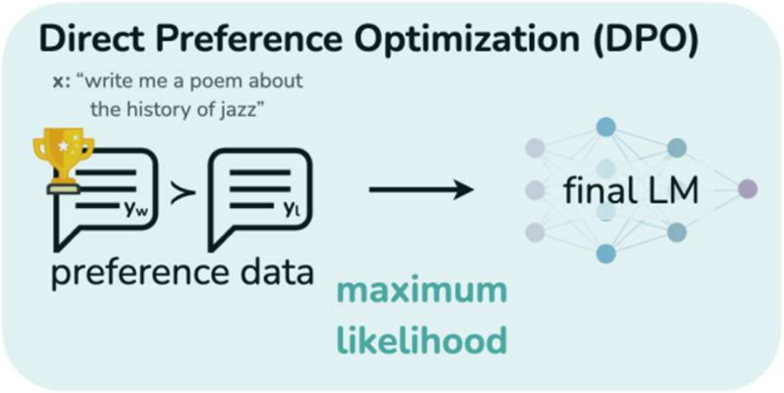


Ref: Lei et al, "LeVo: High-quality song generation with multi-preference alignment," arXiv **2025.09** / NeurIPS 2025

DPO Post-tuning

- **DPO**: direct preference optimization (NeurIPS'23)





- Generating the preference data here
  - lyric alignment preferences (through ASR)
  - prompt consistency preferences (through MuQ-MuLan, or CLAP)
  - musicality preferences (through crowdsourcing rating, or SongEval)

Ref: Lei et al, "LeVo: High-quality song generation with multi-preference alignment," arXiv **2025.09** / NeurIPS 2025

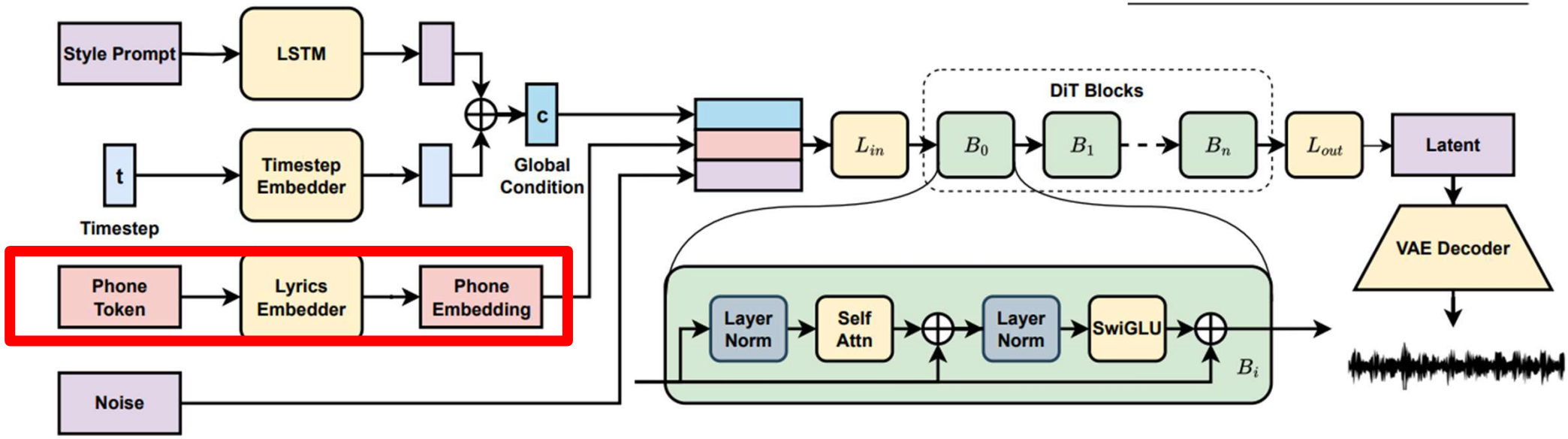
DiffRhythm

https://nzqian.github.io/DiffRhythm/

Method	RTF	Relative Speed
Yue	0.083x	1.00x
DiffRhythm	10.03x	120.84x

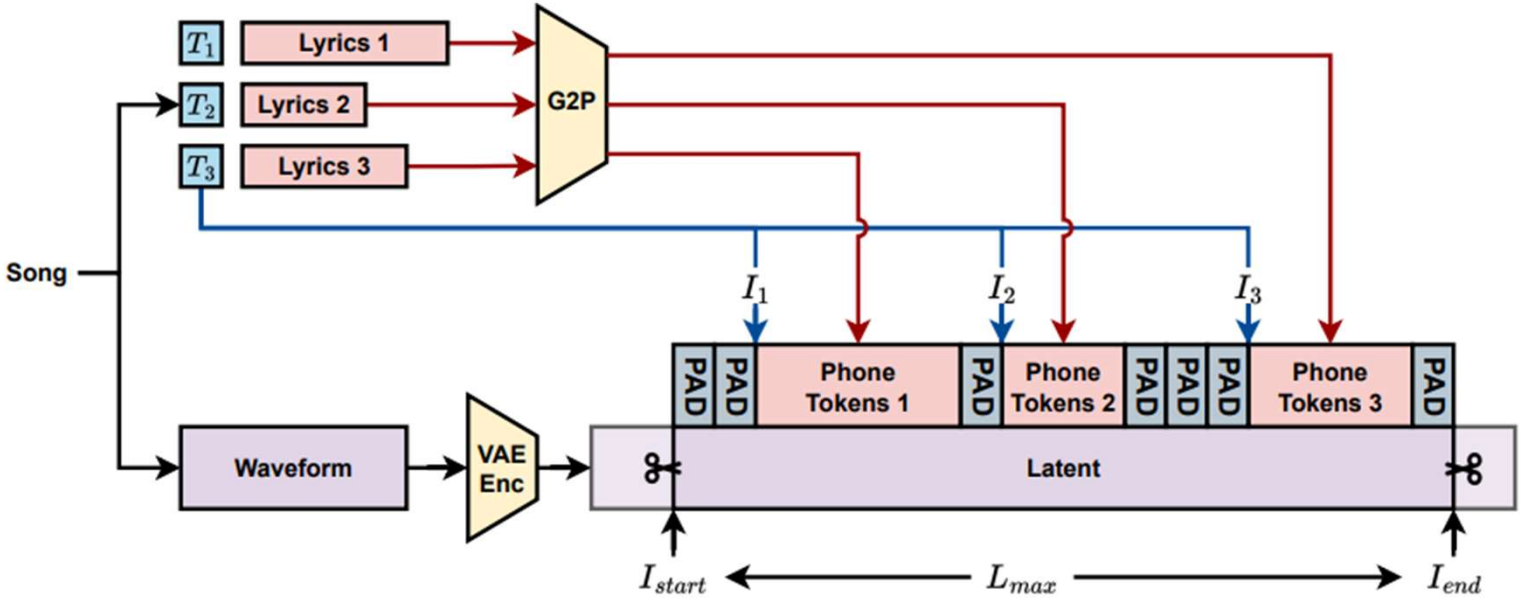
- Generate the mixture directly
- Faster: <0.04 RTF

Generation Length	RTF↓
285 s	0.034
95 s	0.037



Ref: Ning et al, "DiffRhythm: Blazingly fast and embarrassingly simple end-to-end full-length song generation with latent diffusion," arXiv **2025.03**

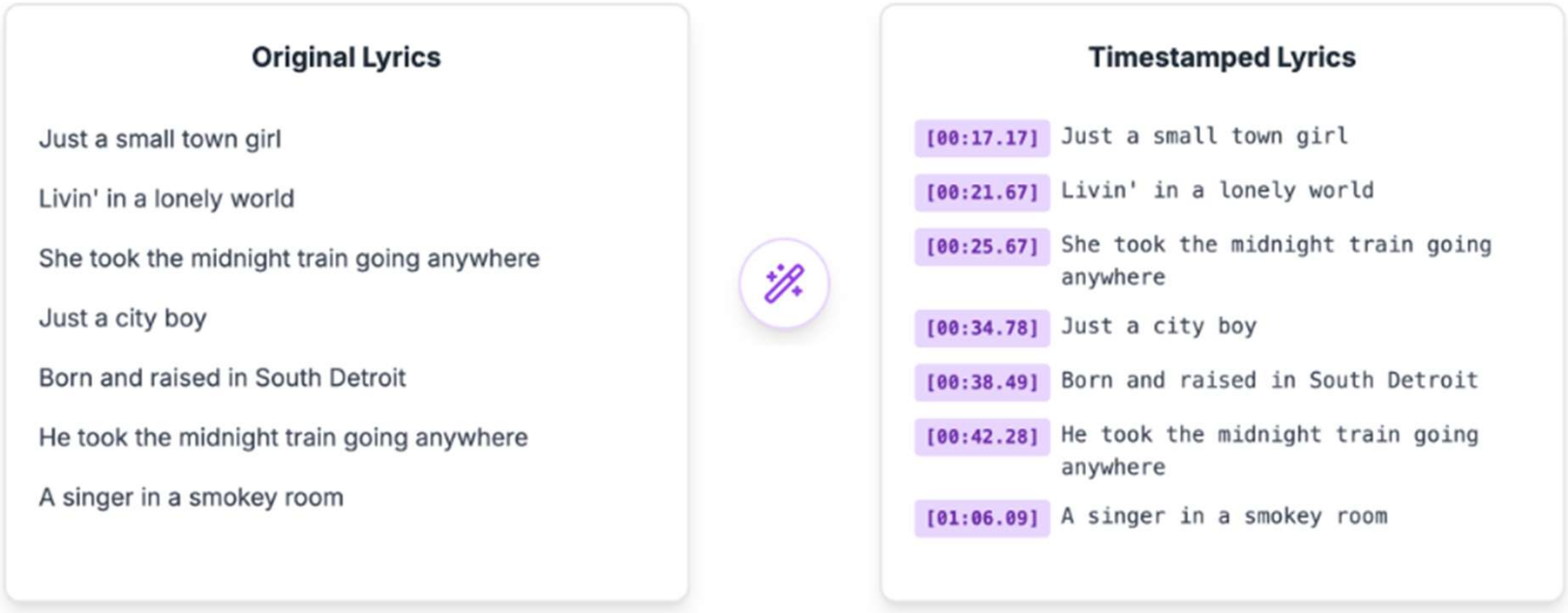
DiffRhythm



- “With conventional text conditioning approaches in diffusion-based TTS models like cross-attention mechanisms or direct feature concatenation, we failed to achieve intelligibility in song generation.”
- Propose to reduce the difficulty of alignment with a sentence-level alignment paradigm that requires only sentence-start annotations

Lyrics with Sentence-level Timestamps (*.LRC)

- Fetch from the Internet
- Or use an automatic lyrics transcription model

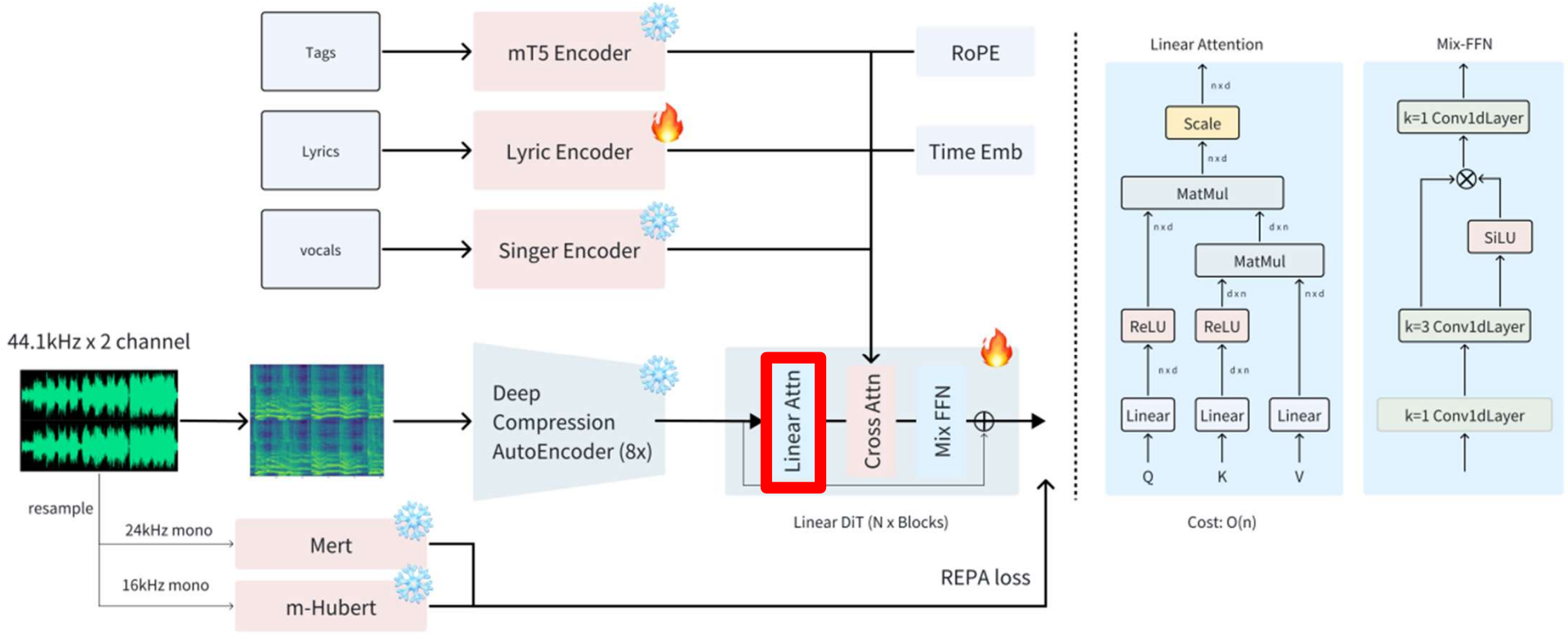


(https://www.reddit.com/r/SideProject/comments/1lsqugy/made_a_free_aipowered_lrc_generator_sync/)

ACE-Step

https://ace-step.github.io/

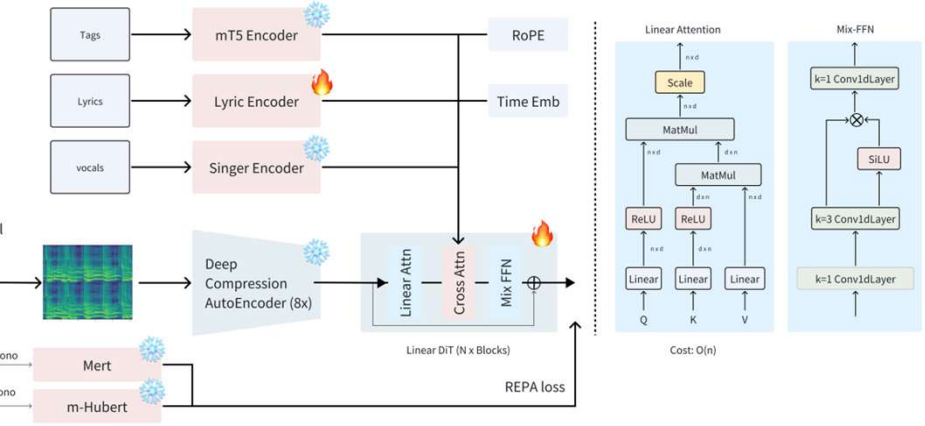
Method	RTF	Relative Speed
Yue	0.083x	1.00x
DiffRhythm	10.03x	120.84x
ACE-Step	15.63x	188.31x

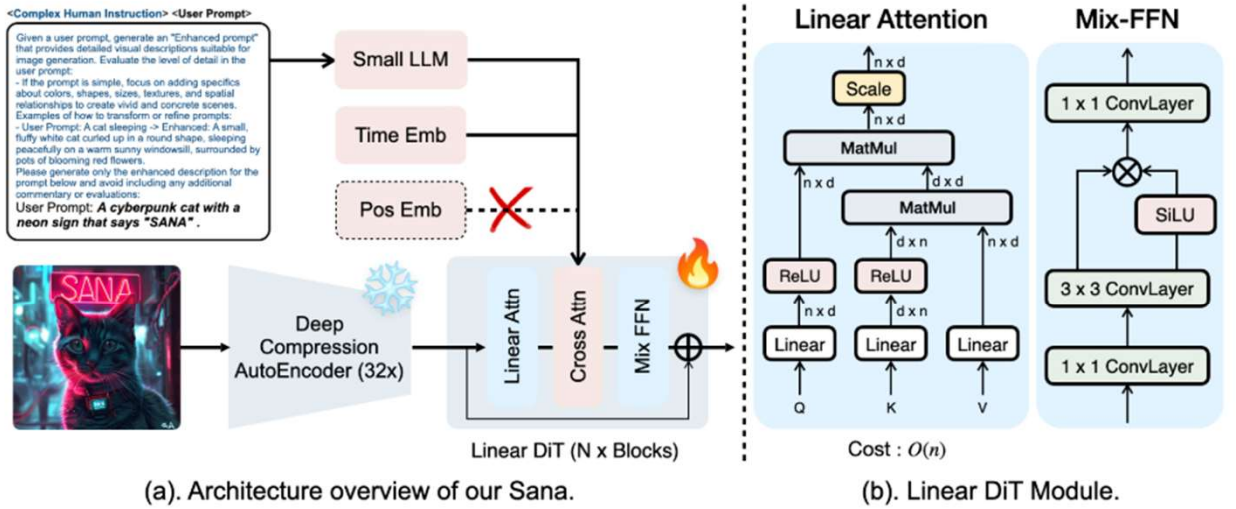


Ref: Gong et al, "ACE-Step: A step towards music generation foundation model," arXiv **2025.05**

ACE-Step

- Adapt the **Linear DiT** structure from NV's text-to-image model SANA with two modifications
  - a single AdaLN layer's parameters are shared across all DiT blocks
  - Conv-1D instead of Conv-2D





Overview of Sana: Fig. (a) describes the high-level training pipeline, containing our 32× deep compression Autoencoder, Linear DiT, and complex human instruction. Note that Positional embedding is not required in our framework. Fig. (b) describes the detailed design of the Linear Attention and Mix-FFN in Linear DiT.

SANA: Efficient High-resolution image synthesis with linear diffusion transformers, arXiv 2024
https://github.com/NVlabs/Sana

Ref: Gong et al, "ACE-Step: A step towards music generation foundation model," arXiv **2025.05**

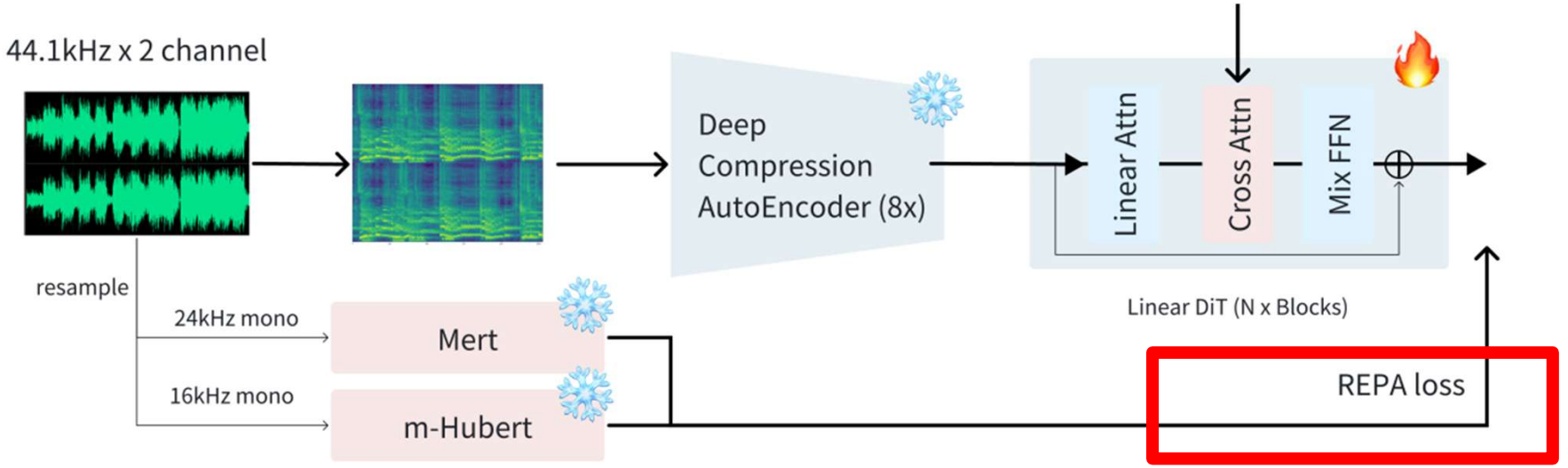
ACE-Step

- Data
  - 1.8 million unique musical pieces (roughly 100,000 hours)
  - Meta's Audiobox aesthetics toolkit for quality control
  - Qwen-omini for audio captioning
  - Whisper 3.0 for lyrics transcription
  - MIR tools for structural segmentation, key detection, tempo estimation
- GPU

Parameter	Specification
Computational Resources	15 nodes, each with 8 NVIDIA A100 GPUs (120 GPUs total).
Batch Size	Per-GPU: 1, Global: 120.
Steps	460,000 steps for pretrain; 240,000 steps for finetune.
Dataset	Full 100,000-hour music dataset for pretrain; Curated high-quality subset of approx. 20,000 hours for fintune
Total Training Time	Approx. 264 hours.
Optimizer Type	AdamW with 1e-2 weight decay and (0.8, 0.9) betas.
Gradient Clipping	Max norm of 0.5.
Learning Rate	1e-4, with linear warm-up over the first 4,000 steps.
Timestep Sampling	Logit-normal scheme: $u \sim \mathcal{N}(\text{mean} = 0.0, \text{std} = 1.0)$; shift=3.0.
Sequence Lengths (Max Context)	4096 tokens for lyrics, 256 tokens for text prompts, 2584 tokens for mel-spectropgram latents

ACE-Step

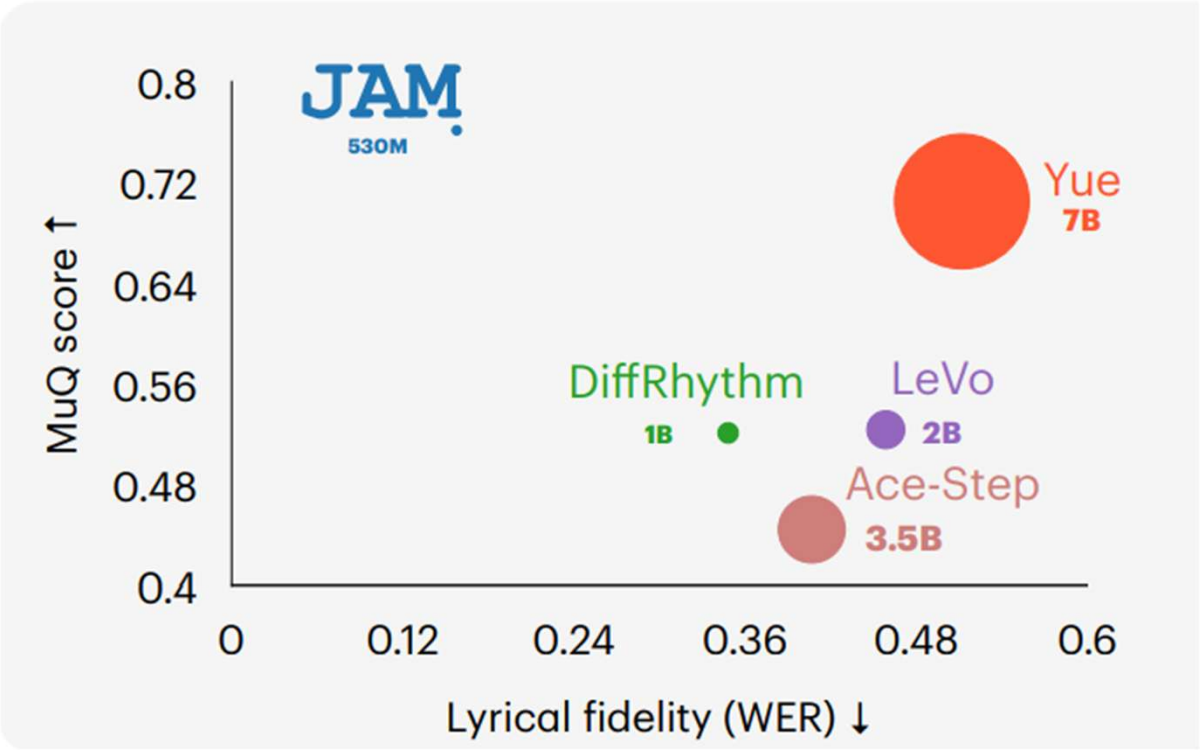
- Addressing lyric-audio alignment challenges
  - using phoneme embeddings alone is not enough
  - the Representation Alignment (**REPA**) framework (ICLR'25) is found critical
    - extract features from an **intermediate Linear DiT layer** of ACE-Step (the 8th layer out of 24)
    - require them to be similar to **MERT** and **HuBERT** embeddings
    - (this is similar to the use of X-Codec in RVQ-based models)



JAM

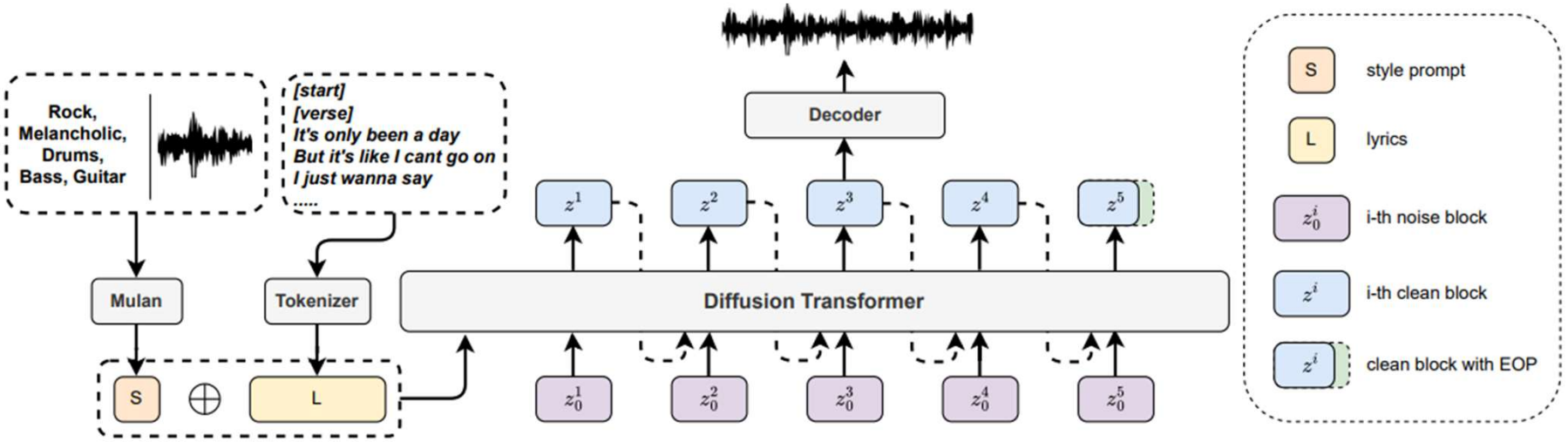
https://declare-lab.github.io/jamify

- Conditional **flow-matching** model (530M parameters)
- Use both **word-** and **phoneme-level timing** inputs (not just sentence-level) for reduced word error rates (WER) and phoneme error rates (PER)
- Use SongEval as the reward signal for **DPO**



Ref: Liu et al, "JAM: A tiny flow-based song generator with fine-grained controllability and aesthetic alignment," arXiv **2025.07**

DiffRhythm 2



- NAR (DiffRhythm) → **Semi-AR** block flow matching (DiffRhythm 2)
- EOP (end-of-prediction) mechanism, stochastic block REPA loss, cross-pair preference optimization, ultra-low 5Hz music VAE
- Sentence timestamps not needed any more

Ref: Jiang et al, "DiffRhythm 2: Efficient and high fidelity song generation via block flow matching," arXiv **2025.10**

ACE-Step 1.5

https://github.com/ace-step/ACE-Step-1.5

Table 1: Comparison with commercial (top) and open-source (bottom) music generation models. The evaluation configuration is 1.7B LM + dynamic distilled DiT (no RL). **Bold** = best, underline = second best. ↑ higher is better.

Model	AudioBox ↑				SongEval ↑					Style Align ↑	Lyric Align ↑
	CE	CU	PC	PQ	Coh.	Mus.	Mem.	Cla.	Nat.		
Udio-v1.5	7.45	7.65	6.15	8.03	4.15	3.96	4.09	3.93	3.86	34.9	24.8
Suno-v4.5	7.63	7.85	6.22	8.25	4.64	4.51	4.63	4.53	4.49	40.5	32.7
Suno-v5	7.69	7.87	**6.51**	8.29	**4.72**	4.62	4.71	4.63	4.56	**46.8**	**34.2**
Mureka-V7.6	7.44	7.71	6.35	8.13	4.43	4.29	4.35	4.29	4.21	36.2	22.4
MinMax-2.0	**7.71**	7.95	6.42	**8.38**	4.61	4.51	4.59	4.50	4.41	43.1	29.5
Yue [18]	6.58	7.29	4.95	7.39	3.01	2.80	2.85	2.79	2.82	26.8	−4.6
ACE-Step 1.0	7.22	7.52	6.50	7.76	3.99	3.73	3.85	3.78	3.68	28.5	0.9
LeVo [19]	7.61	7.78	5.92	8.31	3.55	3.35	3.32	3.31	3.20	29.4	−1.2
DiffRhythm 2 [20]	7.25	7.61	6.33	7.99	3.99	3.79	3.97	3.82	3.66	32.1	3.8
HeartMuLa [21]	7.66	7.89	6.15	8.25	4.68	4.55	4.69	4.55	4.45	31.7	28.6
ACE-Step 1.5	7.42	**8.09**	6.47	8.35	**4.72**	**4.67**	**4.72**	**4.66**	**4.59**	39.1	26.3

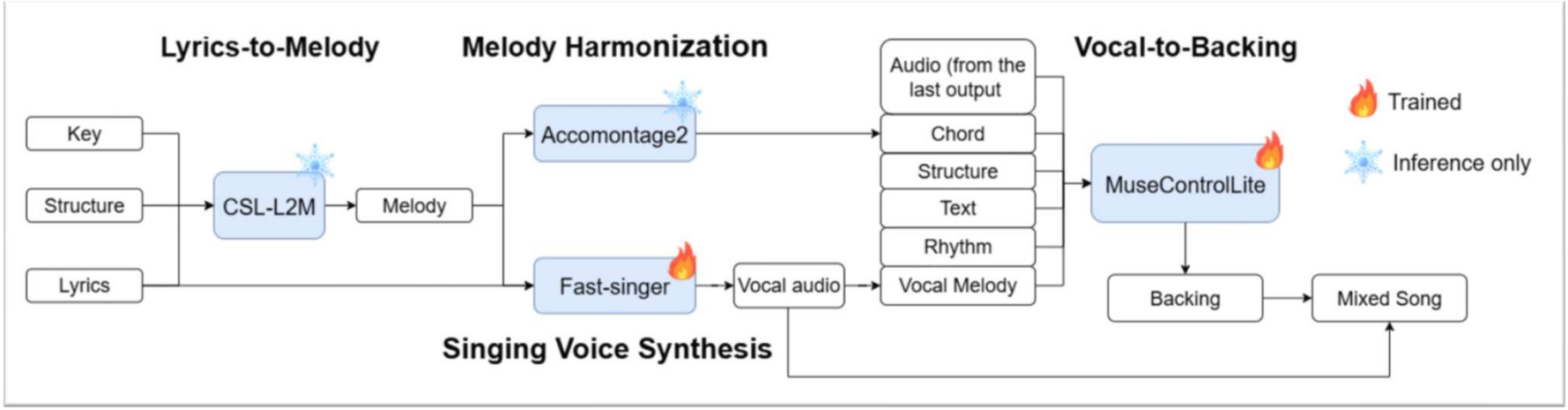
Ref: Gong et al, "ACE-Step 1.5: Pushing the boundaries of open-source music generation," arXiv **2026. 01**

MIDI-SAG

https://composerflow.github.io/web/

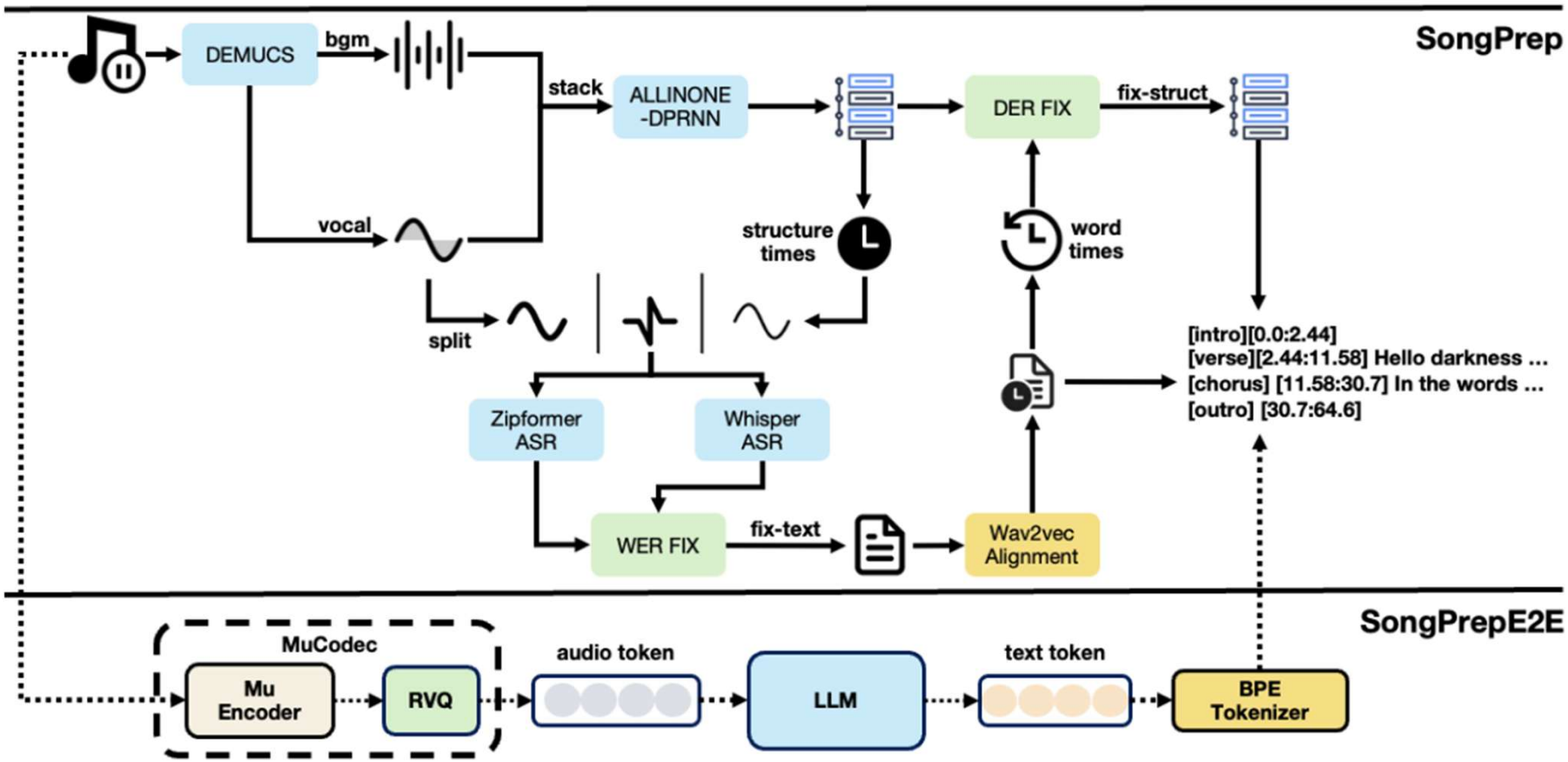
System	Training data (hrs)	Compute resources
Yue (Yuan et al., 2025)	650K	16× H800
DiffRhythm (Ning et al., 2025)	60K	8× Huawei Ascend 910B
Acestep (Gong et al., 2025)	100K	120× A100
SongBloom (Yang et al., 2025)	100K	16× A100
JAM (Liu et al., 2025a)	54K	8× H100
Levo (Lei et al., 2025)	110K	8× A100
ComposerFlow	**6K**	**1 × 3090**

- Compositional pipeline
  - Low-cost training, interpretability/editability
- Lyrics → **MIDI** → audio
- Specifically use **SVS** for better lyrics control



Ref: Tsai et al, "MIDI-informed singing accompaniment generation in a compositional song pipeline," arXiv **2026. 02**

Data Preparation Pipeline: SongPrep



Ref: Tan et al, “SongPrep: A preprocessing framework and end-to-end model for full-song structure parsing and lyrics transcription,” arXiv 2025.09

Data Preparation Pipeline: SongPrep

https://github.com/stppaing126/SongPrep

- Multistage pipeline can be slow
- Build an E2E model by finetuning an LALM model (Qwen2-7B)
  - SongPrepE2E can process a complete song up to four minutes and directly output its structured lyrics
  - Even better WER thanks to the pretrained language capabilities

Table 2. Performance of the pipeline

Model	Data Filter	DER ↓	WER ↓	RTF ↓
SongPrep	-	**15.8%**	27.7%	0.235
SongPrepE2E	wer <0.3	18.1%	**24.3%**	**0.108**
SongPrepE2E	wer <0.1	18.8%	27.3%	**0.108**



Ref: Tan et al, "SongPrep: A preprocessing framework and end-to-end model for full-song structure parsing and lyrics transcription," arXiv 2025.09

YuE2

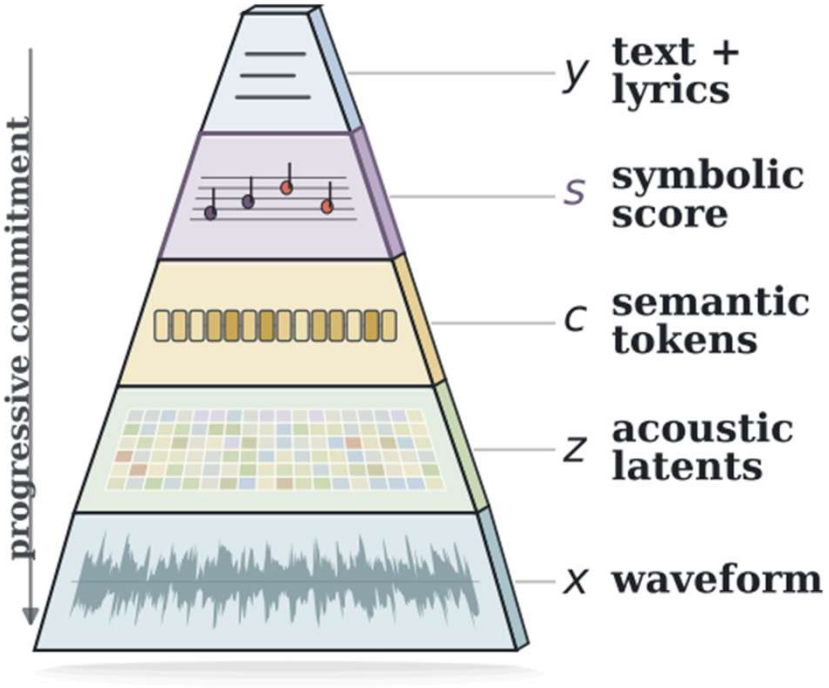
https://github.com/multimodal-art-projection/YuE

- Symbolic planning (**MIDI**, **then audio**)
  - Like MIDI-SAG
- Use **semantic tokens** (from MERT2)

$$x \xrightarrow{\text{SheetSage2}} s,$$

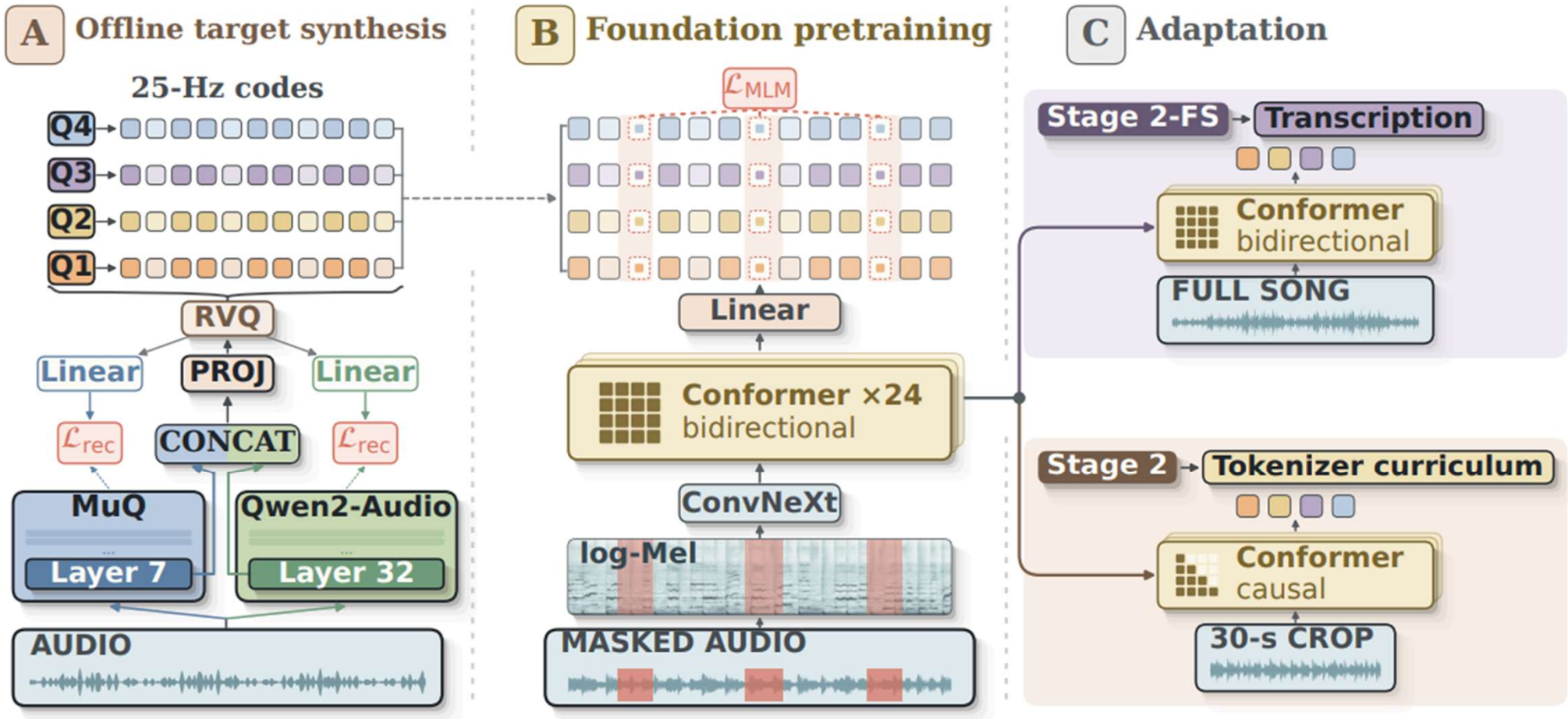
$$x \xrightarrow{\text{MERT2 tokenizer}} c,$$

$$x \xrightarrow{\text{VAE}} z.$$



Ref: Yuan et al, "YuE2: Unifying symbolic and audio music generation at frontier quality," arXiv **2026.09**

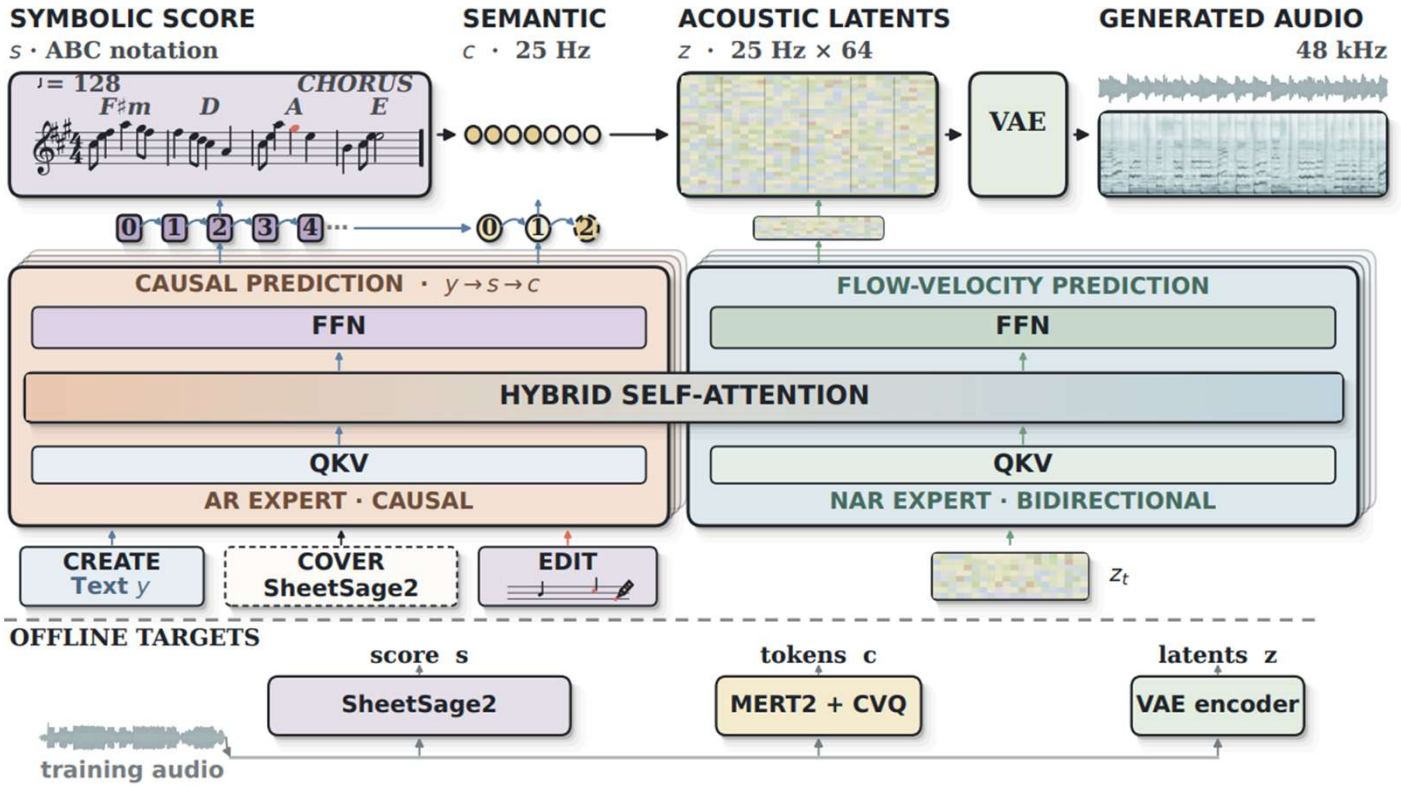
YuE2 / MERT2 / SheetSage 2



Ref: Yuan et al, "YuE2: Unifying symbolic and audio music generation at frontier quality," arXiv **2026.09**

YuE2

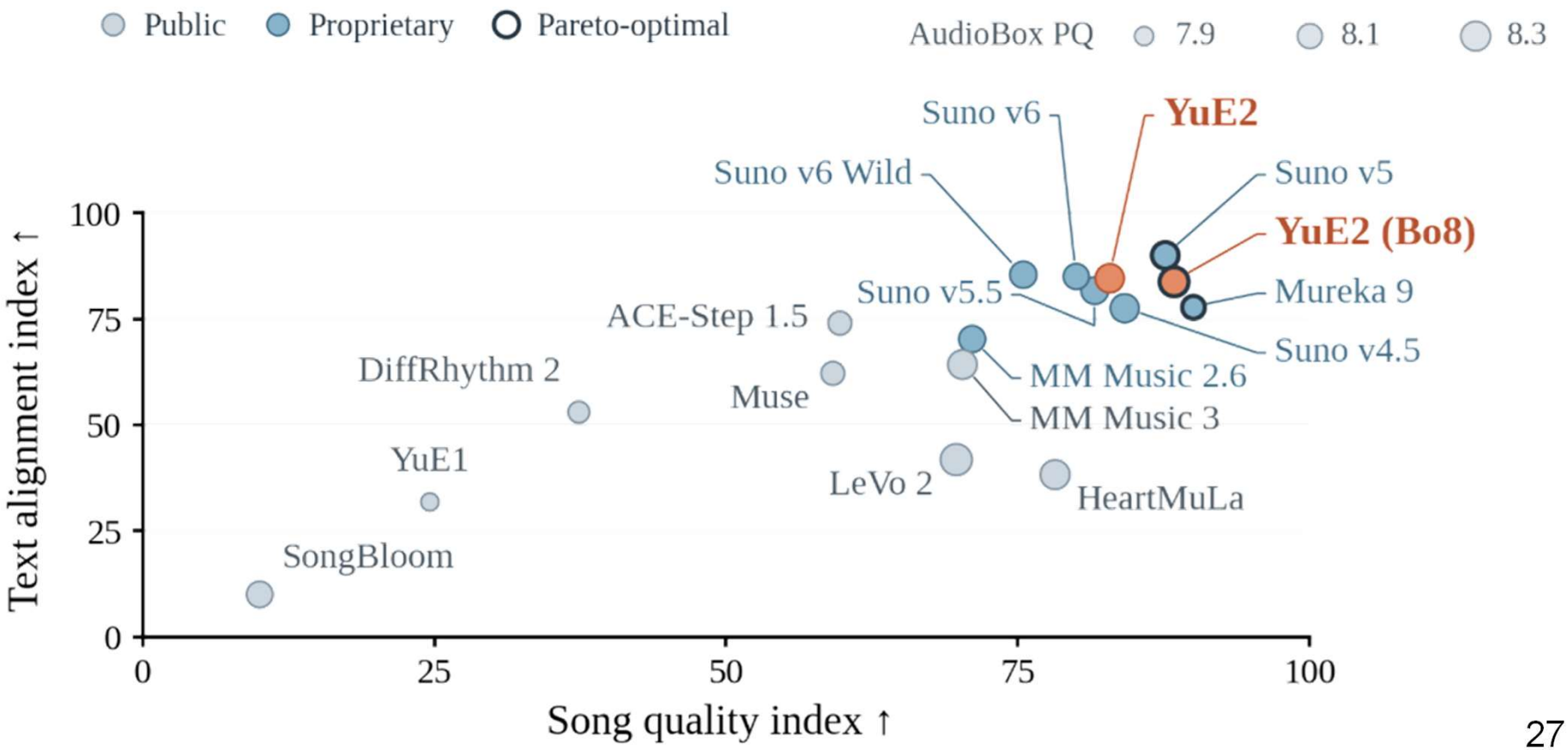
- AR-NAR Mixture-of-Transformers (MoT)



Ref: Yuan et al, "YuE2: Unifying symbolic and audio music generation at frontier quality," arXiv **2026.09**

Open Source Models for Song Generation

- Current SOTA: **YuE2** & **ACE-Step 1.5**



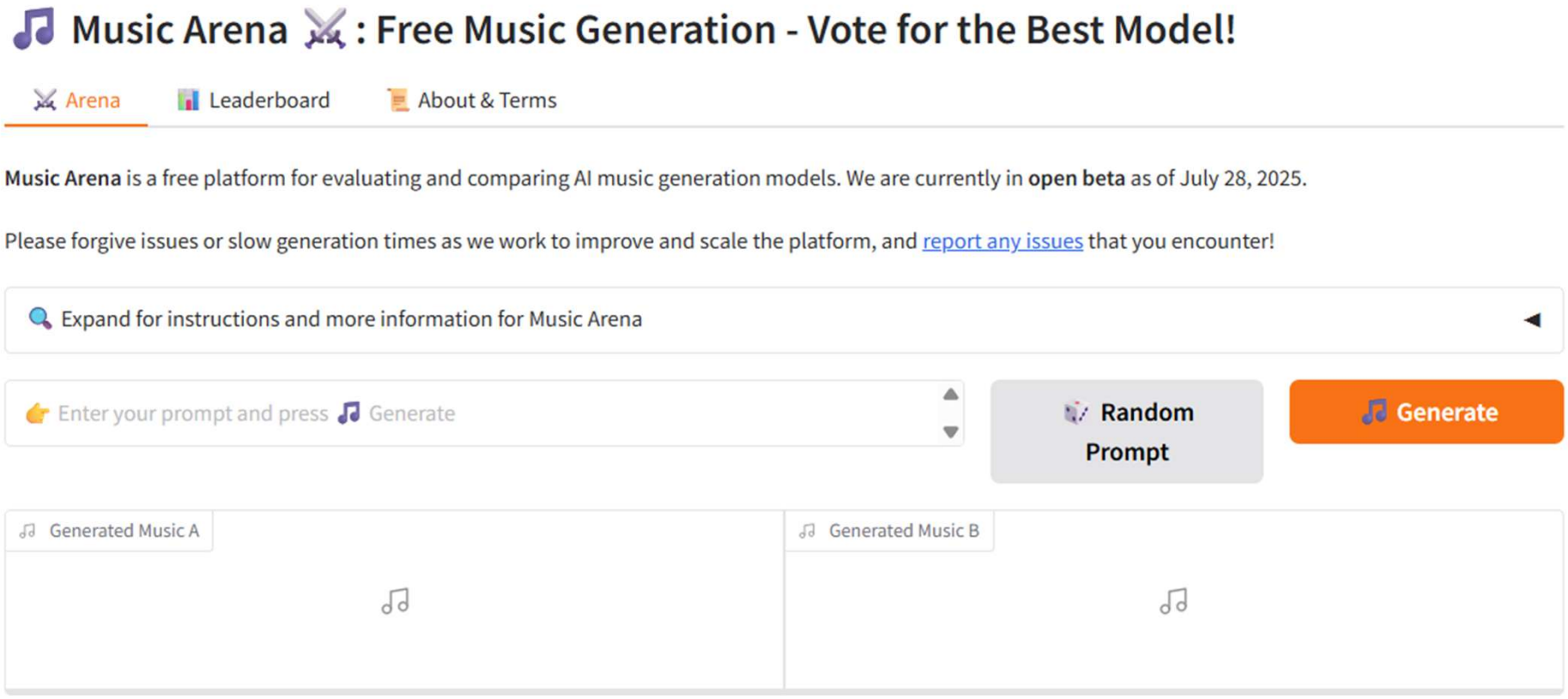
Song Generation: AR vs NAR

- **AR**
  - YuE / YuE2
  - Levo / Levo 2
  - HeartMuLa
  - SongBloom
- **NAR**
  - DiffRhythm / DiffRhythm2
  - ACE-Step / ACE-Step 1.5
  - Muse
  - JAM
  - MIDI-SAG

- **AR**: Higher quality
- **NAR**: Superior speed

Music Arena: Live, Blind, Side-by-side Human Evaluation

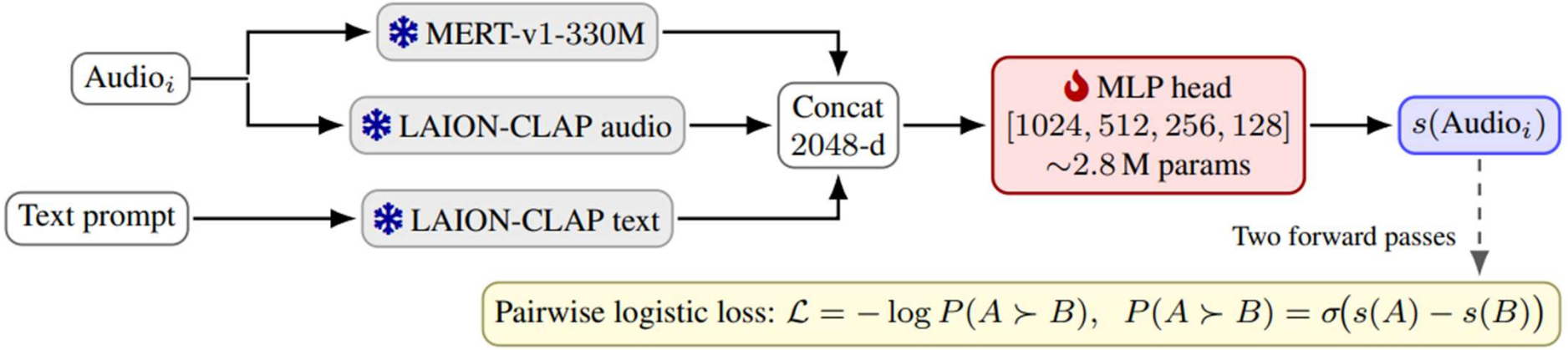
https://beta.music-arena.org/



Ref: Kim et al, “Music Arena: Live evaluation for text-to-music,” NeurIPS 2025

TuneJury

	Framework	Input	Output	Supervision
TuneJury (ours)	RankNet pairwise	TA	1-d scalar	∼17.5 K human-rated pairs
CMI-RM [12]	RankNet pairwise	TLRA	2-d (align, qual)	∼6.6 K human + ∼110 K pseudo
SongEval-RM [16]	MOS regression	A	5-d aesthetic	SongEval MOS
Audiobox-Aesthetics [19]	MOS regression	A	4-d aesthetic	Audiobox MOS
MuQ-Eval [20]	MOS regression	A	2-d (align, qual)	MusicEval MOS
PAM score [17]	Zero-shot audio-LM	TA	1-d scalar	zero-shot



Ref: Kim et al, “TuneJury: An open metric for improving music generation preference alignment,” arXiv **2026.06**